Chapter 14: Sorting Algorithms

EECS 281 notes, condensed. Covers 14.1 to 14.11.

Legend

- **BONUS** = marked as bonus material per course directions (Section 14.10, Radix Sort). Included for completeness, low priority.
- Everything else in this chapter is covered in class and is testable.
- Boxes: **blue** = definition, **green** = worked example, **yellow** = remark / detail.

14.1 Types of Sorting Algorithms

- **Elementary sorts** (bubble, selection, insertion): simple, easy to implement, usually worse asymptotic complexity than high-performance sorts.
- Elementary sorts are still useful because:
  - They are good enough when the data set is small.
  - They are used as building blocks inside advanced sorting algorithms.
- Order of coverage: elementary sorts, then advanced comparison sorts (heap, quick, merge), then linear-time sorts (counting, radix) which need special assumptions about the data.

Inversion: a pair of values in a sequence that is *not* in sorted order.
Formally, for array `arr` and indices `i < j`, the pair `(arr[i], arr[j])` is an inversion if `arr[i] > arr[j]` (assuming ascending order is wanted).
`Example: [0, 1, 2, 3, 5, 4, 6, 7]  →  5 and 4 form one inversion.`
Nearly sorted = few inversions. Reverse sorted = maximum inversions.

Stable sort: the relative order of *equivalent* elements is unchanged by the sort. If A and B are equivalent and A comes before B before the sort, A is still before B after the sort.

Example (stability): sorting only by the number key.
`before: {14,"Banana"} {12,"Carrot"} {14,"Apple"} {15,"Eggplant"} {13,"Durian"}`
`after : {12,"Carrot"} {13,"Durian"} {14,"Banana"} {14,"Apple"} {15,"Eggplant"}`
"Banana" was before "Apple" originally, so a stable sort keeps it before "Apple".

Adaptive sort: the sequence of operations performed depends on the order of the input data. Adaptive sorts take advantage of existing order (for example, they run faster on nearly sorted arrays). Because of this, adaptive sorts often have different best-case and worst-case time complexities.

Non-adaptive sort: always performs the same sequence of operations regardless of input order. Typically simpler to implement, since no order checking is done.

What determines efficiency of a sort

- **Asymptotic time complexity**, measured by counting comparisons and swaps.
- **Auxiliary space usage.** Some sorts are **in-place** (Theta(1) extra memory), others need extra memory that scales with input size, either through an extra container or through recursive stack frames.

14.2 Bubble Sort

- Repeatedly compares two **adjacent** elements and swaps them if they are out of order.
- At the end of each pass, the largest remaining element "bubbles" to the end of the array and is fixed there.

```
void bubble_sort(std::vector<int32_t>& vec) {
  for (size_t i = 0; i < vec.size() - 1; ++i) {
    for (size_t j = 0; j < vec.size() - i - 1; ++j) {
      if (vec[j] > vec[j + 1]) {          // out of order, swap
        std::swap(vec[j], vec[j + 1]);
      } // if
    } // for j
  } // for i
} // bubble_sort()
```

Example on [5, 3, 9, 1, 7, 8, 2, 4, 6]
Pass 1 (compare each adjacent pair left to right)
`[5,3,...]  5>3 swap → [3,5,9,1,7,8,2,4,6]`
`5 vs 9   in order, no swap`
`9 vs 1 swap → [3,5,1,9,7,8,2,4,6]`
`9 vs 7 swap → [3,5,1,7,9,8,2,4,6]`
`9 vs 8 swap → [3,5,1,7,8,9,2,4,6]`
`9 vs 2 swap → [3,5,1,7,8,2,9,4,6]`
`9 vs 4 swap → [3,5,1,7,8,2,4,9,6]`
`9 vs 6 swap → [3,5,1,7,8,2,4,6,9]` pass 1 done, **9** is fixed

Pass 2 (ignore the fixed 9)
`3 vs 5 no swap; 5 vs 1 swap → [3,1,5,7,8,2,4,6,9]`
`5 vs 7 no; 7 vs 8 no; 8 vs 2 swap; 8 vs 4 swap; 8 vs 6 swap`
`result [3,1,5,7,2,4,6,8,9]` pass 2 done, **8** is fixed
Repeat until the array is fully sorted.

Complexity of the basic version

- Pass 1 does n-1 comparisons, pass 2 does n-2, and so on.
- Total = (n-1) + (n-2) + ... + 1 = n(n-1)/2 = **Theta(n^2)** in every case.

Remark: This is not the only way to write bubble sort. You can also iterate from back to front, in which case the **smallest** values get bubbled to the front of the array.

Adaptive bubble sort

- Add a flag: if **no swaps** were made during a whole pass, the array must already be sorted, so break out immediately.

```
// adaptive bubble sort
void bubble_sort(std::vector<int32_t>& vec) {
  for (size_t i = 0; i < vec.size() - 1; ++i) {
    bool swap_made = false;
    for (size_t j = 0; j < vec.size() - i - 1; ++j) {
      if (vec[j] > vec[j + 1]) {
        std::swap(vec[j], vec[j + 1]);
        swap_made = true;
      } // if
    } // for j
    // no swaps during this pass means we are sorted
    if (!swap_made) {
      break;
    } // if
  } // for i
} // bubble_sort()
```

Example on already sorted [1, 2, 3, 4, 5]: 1 vs 2, 2 vs 3, 3 vs 4, 4 vs 5 are all in order, so `swap_made` stays false and the sort terminates after a single pass. Best case becomes **Theta(n)**.

Properties

- **Stable:** yes, as long as equality does *not* cause a swap (use `>`, not `>=`).
- **Adaptive:** yes, with the swap flag.
- **In place:** yes, Theta(1) auxiliary space.

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n)	Theta(n^2)	Theta(n^2)	Theta(1)	**Yes**	**Yes**

14.3 Selection Sort

- Repeatedly **selects the smallest unsorted element** and swaps it into its correct sorted position.

```
void selection_sort(std::vector<int32_t>& vec) {
  for (size_t i = 0; i < vec.size() - 1; ++i) {
    // find the smallest element not yet in sorted position
    size_t min_index = i;
    for (size_t j = i + 1; j < vec.size(); ++j) {
      if (vec[j] < vec[min_index]) {
        min_index = j;
      } // if
    } // for j
    std::swap(vec[i], vec[min_index]);   // put it in place
  } // for i
} // selection_sort()
```

Example on [5, 3, 9, 1, 7, 8, 2, 4, 6] (bold = fixed in sorted position)

```
[5,3,9,1,7,8,2,4,6]   1 is smallest, swap to index 0
[1,3,9,5,7,8,2,4,6]   2 is next smallest, swap to index 1
[1,2,9,5,7,8,3,4,6]   3 next, swap to index 2
[1,2,3,5,7,8,9,4,6]   4 next, swap to index 3
[1,2,3,4,7,8,9,5,6]   5 next, swap to index 4
[1,2,3,4,5,6,9,7,8]   6 next, swap to index 5
[1,2,3,4,5,6,7,9,8]   7 next, swap to index 6
[1,2,3,4,5,6,7,8,9]   8 next, swap to index 7
[1,2,3,4,5,6,7,8,9]   last element is already in place, done
```

Complexity and properties

- n-1 comparisons on the first pass, n-2 on the second, and so on, giving **Theta(n^2)** in **all** cases.
- You can add a check to avoid swapping an element with itself, but that does not remove the Theta(n^2) comparisons.
- **Not adaptive:** execution is independent of the input order.
- **Not stable:** you cannot predict where an element lands after a swap.

Why selection sort is not stable. Take [4, 3, 4', 2, 1] where 4' is the second 4.

```
[4, 3, 4', 2, 1]   1 is smallest, swap to index 0
[1, 3, 4', 2, 4]   the first 4 got thrown to the back, behind 4'
...
[1, 2, 3, 4', 4]   4' now precedes 4, so the sort is not stable
```

The first 4 could have landed before or after 4' depending on where the 1 was, so the outcome is unpredictable.

Note: It is technically possible to make selection sort stable by changing its behavior or using extra memory, but for this class "selection sort" means the standard, unstable implementation unless stated otherwise.

The redeeming quality of selection sort

- **It minimizes the number of swaps.** Selection sort needs at most **n-1 swaps** (one per iteration). Bubble sort could need a swap for every pair.
- Do not confuse swap count with runtime: even with zero swaps, selection sort still does Theta(n^2) **comparisons**, which is why its best case is still Theta(n^2).

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n^2)	Theta(n^2)	Theta(n^2)	Theta(1)	**No**	**No**

14.4 Insertion Sort

- Looks at each element in order and builds the sorted result one element at a time.
- The current element is compared only with the elements **before** it. Every element that is out of position relative to the current element (that is, forms an inversion with it) gets shifted one slot right, then the current element is inserted into the gap.

```
void insertion_sort(std::vector<int32_t>& vec) {
  for (size_t i = 1; i < vec.size(); ++i) {
    int32_t current = vec[i];
    int32_t j = i - 1;
    while (j >= 0 && vec[j] > current) {
      vec[j + 1] = vec[j];    // shift right
      --j;
    } // while
    vec[j + 1] = current;     // insert
  } // for i
} // insertion_sort()
```

Example on [5, 3, 9, 1, 7, 8, 2, 4, 6]

```
[5,3,9,1,7,8,2,4,6]   5 is trivially in position
consider 3: goes before 5, shift 5 right → [3,5,9,1,7,8,2,4,6]
consider 9: already after 3 and 5, no shifting → [3,5,9,1,7,8,2,4,6]
consider 1: goes before 3, shift 9, 5, 3 right → [1,3,5,9,7,8,2,4,6]
consider 7: goes before 9, shift 9 right → [1,3,5,7,9,8,2,4,6]
consider 8: goes before 9, shift 9 right → [1,3,5,7,8,9,2,4,6]
consider 2: goes before 3, shift 9,8,7,5,3 right → [1,2,3,5,7,8,9,4,6]
consider 4: goes before 5, shift 9,8,7,5 right → [1,2,3,4,5,7,8,9,6]
consider 6: goes before 7, shift 9,8,7 right → [1,2,3,4,5,6,7,8,9]   sorted
```

Sentinel optimization

- The loop condition `while (j >= 0 && vec[j] > current)` checks two things Theta(n^2) times.
- The `j >= 0` check rarely returns false. It only matters when the current element is the smallest seen so far (must go all the way to index 0).
- **Fix:** first move the overall smallest element to index 0. That element is called a **sentinel value**. Then `vec[j] > current` alone is enough to stop the loop, so only one check per iteration is needed:

```
while (vec[j] > current) { ... }
```

Properties

- **Adaptive:** yes. On an already sorted array, each element needs only one comparison to see it is in place, so only n comparisons total, giving best case **Theta(n)**.
- **Stable:** yes. Elements are considered front to back, and shifting only happens when the current element is **strictly less than** the element it is compared to, so duplicates keep their original relative order.
- **In place:** yes, Theta(1) auxiliary space.
- **Fast in practice for small inputs.** Some standard library implementations of `std::sort()` switch to insertion sort when fewer than 16 elements remain, because it beats the high-performance sorts at that size.

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n)	Theta(n^2)	Theta(n^2)	Theta(1)	**Yes**	**Yes**

Improved insertion sort with sentinel

```
void insertion_sort(std::vector<int32_t>& vec) {
  // find minimum element and place at index 0 (sentinel)
  int32_t min_index = 0;
  for (size_t i = 1; i < vec.size(); ++i) {
    if (vec[i] < vec[min_index]) {
      min_index = i;
    } // if
  } // for i
  std::swap(vec[0], vec[min_index]);
  // run insertion sort on remaining elements
  for (size_t i = 2; i < vec.size(); ++i) {
    int32_t current = vec[i];
    int32_t j = i - 1;
    while (vec[j] > current) {      // only one check now
      vec[j + 1] = vec[j];
      --j;
    } // while
    vec[j + 1] = current;
  } // for i
} // insertion_sort()
```

14.5 Heapsort

- First of the **high-performance sorts**. These are more complex but more efficient for large inputs.
- Heapsort uses the heap structure. Steps:
  1. **Heapify** the whole array into a binary **max-heap** using bottom-up heapify (worst-case Theta(n)).
  2. The largest element is now at the top. **Swap it with the last element** of the array. That places it in its correct sorted position, and it is then ignored.
  3. **Fix down** on the element that was just swapped to the top, restoring the max-heap over the remaining (unfixed) portion.
  4. Repeat steps 2 and 3 until the array is fully sorted.

Example on [53, 17, 89, 46, 34]

```
Start (as a tree, level order): 53 / 17, 89 / 46, 34    array: [53, 17, 89, 46, 34]

Bottom-up heapify: fix down on 17 (swap with 46), then fix down on 53 (swap with 89)
→ max-heap: [89, 46, 53, 17, 34]

89 is largest, swap with last element 34 → [34, 46, 53, 17, 89]   (89 fixed)
fix down on 34 (swaps with 53) → [53, 46, 34, 17, 89]
53 is next largest, swap with 17 → [17, 46, 34, 53, 89]
fix down on 17 (swaps with 46) → [46, 17, 34, 53, 89]
46 next, swap with 34 → [34, 17, 46, 53, 89]
fix down on 34: nothing changes (34 > 17)
34 next, swap with 17 → [17, 34, 46, 53, 89]   sorted
```

```
// fix_down calls fix down on vec[idx], using valid indices in the range [1, numElts)
void fix_down(std::vector<int32_t>& vec, size_t idx, size_t numElts);

void heapsort(std::vector<int32_t>& vec) {
  // build heap using bottom-up heapify
  for (size_t i = vec.size() / 2; i > 0; --i) {
    fix_down(vec, i, vec.size());
  } // for i
  // loop from back to front, assumes dummy element at idx 0 (1-indexing)
  for (size_t j = vec.size(); j > 0; --j) {
    std::swap(vec[1], vec[j]);   // move largest element to the back
    fix_down(vec, 1, j - 1);     // fix top, ignoring already-fixed elements
  } // for j
} // heapsort()
```

Complexity

- Step 1, bottom-up heapify: **Theta(n)**.
- Step 2, fix down is called n times, each taking Theta(log(n)): **Theta(n log(n))**.
- Consecutive steps, so total = Theta(n + n log(n)) = **Theta(n log(n))** in all cases.

Properties

- **Not stable:** heap operations such as fix down give no guarantee about the ordering of duplicates.
- **Not adaptive:** hard to make adaptive for similar reasons.
- **In place:** Theta(1) auxiliary space.
- Heapsort's Theta(n log(n)) guarantee **relies on random access**, so it does not apply to linked lists.

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n log(n))	Theta(n log(n))	Theta(n log(n))	Theta(1)	**No**	**No**

14.6 Quicksort

14.6.1 Quicksort Partitioning

- Quicksort is a **divide-and-conquer** algorithm built on **partitioning**. Repeat until sorted:
  1. Select a **pivot** element. (These notes always select the **last** element of the range.)
  2. **Partition** the remaining elements so that everything to the left of the pivot is less than the pivot and everything to the right is greater.
- After a partition, the pivot is in its final sorted position. Then recurse on the left subarray and the right subarray.

```
int32_t partition(std::vector<int32_t>& vec, int32_t left, int32_t right) {
  int32_t pivot = --right;
  while (true) {
    while (vec[left] < vec[pivot]) {
      ++left;
    } // while
    while (left < right && vec[right - 1] >= vec[pivot]) {
      --right;
    } // while
    if (left >= right) {
      break;
    } // if
    std::swap(vec[left], vec[right - 1]);
  } // while
  std::swap(vec[left], vec[pivot]);
  return left;
} // partition()

void quicksort(std::vector<int32_t>& vec, int32_t left, int32_t right) {
  if (left + 1 >= right) {
    return;
  } // if
  int32_t pivot = partition(vec, left, right);
  quicksort(vec, left, pivot);
  quicksort(vec, pivot + 1, right);
} // quicksort()
```

- `partition()` takes the index range `[left, right)` and uses the last element as the pivot.
- Two indices are tracked: **L** starts at the first element, **R** starts at the last non-pivot element.
- **Increment L** until it points to a value greater than the pivot. **Decrement R** until it points to a value less than the pivot. Swap those two elements. Repeat.
- When **L and R meet**, swap the element at that position with the pivot. The pivot is now in its correct sorted position.

Example on [5, 3, 9, 1, 7, 8, 2, 4, 6], pivot = 6 (last element)
```
[5, 3, 9, 1, 7, 8, 2, 4, | 6]   L at 5, R at 4
L moves to 9 (first value > 6). R already at 4 (< 6). Swap 9 and 4:
[5, 3, 4, 1, 7, 8, 2, 9, | 6]
L moves to 7, R moves to 2. Swap 7 and 2:
[5, 3, 4, 1, 2, 8, 7, 9, | 6]
L moves to 8. R decrements and now meets L.
L and R meet at 8, so swap 8 with the pivot 6:
[5, 3, 4, 1, 2, 6, 7, 9, 8]
6 is in its final position. Everything left of it is smaller, everything right is larger.
```
Recurse left on [5, 3, 4, 1, 2] with pivot 2:
```
L at 5 (> 2), R at 1 (< 2). Swap → [1, 3, 4, 5, 2 | ...]
L and R both land on 3, so swap 3 with pivot 2 → [1, 2, 4, 5, 3, 6, 7, 9, 8]
Left of 2 is a single element (trivially sorted). Right of 2 is [4, 5, 3] with pivot 3:
L and R both land on 4, swap 4 with 3 → [1, 2, 3, 5, 4, 6, 7, 9, 8]
Recursing right of 3 swaps 4 and 5 → [1, 2, 3, 4, 5, 6, 7, 9, 8]
```
Recurse right of the original pivot 6 on [7, 9, 8] with pivot 8:
```
L lands on 9 and R is also on 9, so swap 9 with pivot 8 → [1,2,3,4,5,6,7,8,9]
One element on each side of 8, both trivially sorted. Array fully sorted.
```

14.6.2 Quicksort Complexity Analysis

- The **partitioning step is Theta(n)**: a single pass of the array using L and R.
- The size of the two recursive calls depends entirely on the **value of the pivot**.

Best case: pivot is always the median

- Both sides have n/2 elements. Recurrence:

```
T(n) = 1                 if n = 0 or 1
T(n) = 2 T(n/2) + n      if n > 1
```

- 2T(n/2) is the two recursive calls, n is the partition step. By Master's Theorem: **Theta(n log(n))**.

Worst case: pivot is always the smallest or largest element

- One side is empty, the other holds all n-1 remaining elements. Recurrence:

```
T(n) = 1                 if n = 0 or 1
T(n) = T(n - 1) + n      if n > 1
```

- By substitution: **Theta(n^2)**.

Average case: Theta(n log(n))

Intuitive argument 1 (middle half).

- If every element is equally likely to be the pivot, there is a **50 percent chance** the pivot lands in the "middle half" of the **sorted values** (the values closest to the median, not the elements physically in the middle of the unsorted array).
- If the pivot lands in that middle half, the **worst split possible is 3-to-1** (3n/4 and n/4). So for half of the calls, at worst:

```
T(n) = T(3n/4) + T(n/4) + n
```

- Recursion tree: the sum of values at each level is the work done at that depth.
  - Level 0: n
  - Level 1: 3n/4 + n/4 = n
  - Level 2: 9n/16 + 3n/16 + 3n/16 + n/16 = n
- So each full level costs Theta(n), and total work = n times the number of levels.
- **Catch:** the two branches shrink at different rates. The T(n/4) branch reaches the base case after about $\log_4(n)$ levels, while the T(3n/4) branch needs about $\log_{4/3}(n)$ levels.
- The deepest layers only have work in the longer branch, so they do **less than** n work. Total work is therefore less than n * $\log_{4/3}(n)$ = **Theta(n log(n))**.

Adding in the bad half of the pivots.

- Only 50 percent of pivots create a 3-1 split or better. Assume the other 50 percent are worst-case pivots, and assume they **alternate**: bad split, 3-1 split, bad split, 3-1 split, ...
- That tree is nearly identical to the previous one, except a 3-1 split now happens only every **other** level. The input size shrinks by a factor of 4/3 every two levels instead of every level.
- So the number of levels doubles: **2 $\log_{4/3}$(n)**. Each level is still Theta(n).
- Total = Theta(n) * 2 $\log_{4/3}$(n) = **Theta(n log(n))**.
- Since the average case cannot be worse than Theta(n log(n)) (shown above) and cannot be better than the best case of Theta(n log(n)), it must be **exactly Theta(n log(n))**.

Intuitive argument 2 (9-1 splits).

- Suppose you are extremely unlucky and every pivot gives a 9-to-1 split (90 percent one side, 10 percent the other).
- Each level still sums to n (for example n/10 + 9n/10 = n, and 81n/100 + 9n/100 + 9n/100 + n/100 = n).
- The longest branch reaches the base case after $\log_{10/9}$(n) levels, so total work = Theta(n) * $\log_{10/9}$(n) = **Theta(n log(n))**.
- On average you do better than a 9-1 split every time, so the average case must also be Theta(n log(n)).

> **Note:** Neither explanation is mathematically rigorous. Formal proofs are significantly more complex. These are meant to give intuition.

Improving pivot selection

- The **best pivot is the median**.
- **Option A: exact median every time** using `std::nth_element()` in Theta(n). This makes the worst case Theta(n log(n)), but it is **never used in practice**: the constant factor in front of n log(n) is large enough that the algorithm is actually slower for reasonable input sizes.
- **Option B (practical): estimate the median by sampling.** Randomly pick three elements and use their median as the pivot. Picking three elements is a constant time operation.
  - Worst case is still Theta(n^2) (you could get unlucky), but the chance is extremely low.
  - Once chosen, **swap the pivot with the last element** so the partition code can still assume the pivot is last.

> **Example:** in [17, 15, 34, 2, 1, 4, 42, 14, 34, 57, 18, 9, 28, 84, 5], the random sample 1, 42, 28 has median **28**, so 28 becomes the pivot and is swapped to the end.

Auxiliary space: Theta(log(n))

- Quicksort is **not tail recursive** (two recursive calls), so it needs stack frames.
- If you always recurse into the **smaller** partition first, the larger partition is handled last and that last call is tail recursive.
- The smaller call can only need up to Theta(log(n)) stack frames (if it needed more, it would not be the smaller call). So stack usage is bounded by **Theta(log(n))** instead of approaching Theta(n).

```
void quicksort(std::vector<int32_t>& vec, int32_t left, int32_t right) {
  if (left + 1 >= right) {
    return;
  } // if
  int pivot = partition(vec, left, right);
  if (pivot - left < right - pivot) {
    quicksort(vec, left, pivot);          // smaller side first
    quicksort(vec, pivot + 1, right);
  } // if
  else {
    quicksort(vec, pivot + 1, right);     // smaller side first
    quicksort(vec, left, pivot);
  } // else
} // quicksort()
```

Properties

- **Not stable** in most implementations, because of how partitioning moves elements around.
- **Not adaptive**, because of the complex structure of the algorithm.

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n log(n))	Theta(n log(n))	Theta(n^2)	Theta(log(n))	**No**	**No**

14.7 Mergesort

14.7.1 The Merge Operation

- Mergesort is also **divide-and-conquer**: split the array into two halves, recursively sort each half, then **merge** the two sorted halves together.

```
void mergesort(std::vector<int32_t>& vec, int32_t left, int32_t right) {
  // base case: fewer than two items
  if (right - left < 2) {
    return;
  } // if
  int32_t mid = left + (right - left) / 2;   // split in half
  mergesort(vec, left, mid);                 // sort left half
  mergesort(vec, mid, right);                // sort right half
  merge(vec, left, mid, right);              // merge the two sorted halves
} // mergesort()
```

The merge step, Theta(n)

1. Keep two indices `i` and `j` pointing at the first element of each sorted half.
2. Initialize a separate output vector of size `vec.size()`.
3. If `vec[i] <= vec[j]`, copy `vec[i]` to the output and increment `i`. Otherwise copy `vec[j]` and increment `j`.
4. Repeat until the output is full. If one half runs out first, append all remaining elements of the other half.

> **Example.** Original [5, 6, 3, 1, 8, 2, 4, 7]. After the two recursive calls, the halves are sorted:
> `[ 1 3 5 6 | 2 4 7 8 ]` `i points at 1, j points at 2, k = next open output slot`
>
> `1 < 2 → output [1], ++i`
> `2 < 3 → output [1,2], ++j`
> `3 < 4 → output [1,2,3], ++i`
> `4 < 5 → output [1,2,3,4], ++j`
> `5 < 7 → output [1,2,3,4,5], ++i`
> `6 < 7 → output [1,2,3,4,5,6], ++i`
> `i has run off the end of the first half, so copy the rest of the second half:`
> `output [1,2,3,4,5,6,7,8]`

```
void merge(std::vector<int32_t>& vec, int32_t left, int32_t mid, int32_t right) {
  int32_t size = right - left;
  std::vector<int32_t> output(size);
  for (int32_t i = left, j = mid, k = 0; k < size; ++k) {
    if (i == mid) {
      output[k] = vec[j++];      // first half done, copy rest of second half
    } // if
    else if (j == right) {
      output[k] = vec[i++];      // second half done, copy rest of first half
    } // else if
    else {
      if (vec[i] <= vec[j]) {
        output[k] = vec[i++];
      } // if
      else {
        output[k] = vec[j++];
      } // else
    } // else
  } // for
  std::copy(output.begin(), output.end(), vec.begin() + left);
} // merge()
```

Why mergesort is stable: the check is `vec[i] <= vec[j]`, so on a tie the element from the **first** half is copied first. Duplicates keep their original relative order.

14.7.2 Mergesort Complexity Analysis

- `merge()` does a single pass over the range: **Theta(n)**.
- Two recursive calls of size n/2 plus a linear merge:

```
T(n) = 1                if n = 1
T(n) = 2 T(n/2) + n     if n > 1
```

- Master's Theorem gives **Theta(n log(n))**.
- Mergesort always splits into halves regardless of input, so **best, average, and worst are all Theta(n log(n))**. It is **not adaptive**.
- **Auxiliary space Theta(n)** because of the output vector used during merging.

Remark (linked lists): the merge operation on a **linked list** can be done with Theta(1) auxiliary space. Instead of copying data, you relink nodes: repeatedly pick the smaller head node and attach it to the output list, keeping only a constant number of pointers (current, next). No auxiliary array is needed.
A **recursive** mergesort still needs Theta(log(n)) space for stack frames. A **bottom-up iterative** mergesort removes that too, achieving full Theta(1) auxiliary space on a linked list.

Why mergesort matters

- Mergesort **does not require random access**. This makes it good for:
  - **Linked list sorting.**
  - **External memory sorting:** when the data is too large to fit in main memory, split it into chunks, sort each, and merge.
  - **Parallel / concurrent sorting:** send segments to different servers, each sorts its own part, then combine the results.

Full mergesort on [5, 6, 3, 1, 8, 2, 4, 7]

```
5 6 3 1 8 2 4 7   (split)
[5 6 3 1] [8 2 4 7]
[5 6] [3 1] [8 2] [4 7]
[5][6] [3][1] [8][2] [4][7]   (merge back up)
[5 6] [1 3] [2 8] [4 7]
[1 3 5 6] [2 4 7 8]
[1 2 3 4 5 6 7 8]
```

Bottom-up (iterative) mergesort

- The version above is **top-down** (recursive). Mergesort can also be written iteratively with nested for loops that simulate the recursion: merge groups of 2, then groups of 4, then groups of 8, and so on.

```
void mergesort(std::vector<int32_t>& vec, int32_t left, int32_t right) {
  for (int32_t size = 1; size <= right - left; size *= 2) {
    for (int32_t i = left; i <= right - size; i += 2 * size) {
      merge(vec, i, i + size, min(i + 2 * size, right));
    } // for i
  } // for size
} // mergesort()
```

Bottom-up on [5, 6, 3, 1, 8, 2, 4, 7]

```
Pass 1 (pairs of 2): merge 5,6 / 3,1 / 8,2 / 4,7 → [5,6, 1,3, 2,8, 4,7]
Pass 2 (groups of 4): merge (5,6,1,3) and (2,8,4,7) → [1,3,5,6, 2,4,7,8]
Pass 3 (group of 8): merge everything → [1,2,3,4,5,6,7,8]
```

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n log(n))	Theta(n log(n))	Theta(n log(n))	Theta(n) *	**Yes**	**No**

* Assumes an array. On a linked list: Theta(log(n)) for top-down recursive mergesort, Theta(1) for bottom-up iterative mergesort.

14.8 Analysis of Comparison Sorts

Comparison sort: a sorting algorithm that orders a container by comparing elements against each other with a comparison operator. All six sorts so far are comparison sorts.

Sort	Best Time	Average Time	Worst Time	Aux Space	Stable?	Adaptive?
Bubble	Theta(n)	Theta(n^2)	Theta(n^2)	Theta(1)	**Yes**	**Yes**
Selection	Theta(n^2)	Theta(n^2)	Theta(n^2)	Theta(1)	**No**	**No**
Insertion	Theta(n)	Theta(n^2)	Theta(n^2)	Theta(1)	**Yes**	**Yes**
Heap	Theta(n log(n))	Theta(n log(n))	Theta(n log(n))	Theta(1)	**No**	**No**
Quick	Theta(n log(n))	Theta(n log(n))	Theta(n^2)	Theta(log(n))	**No**	**No**
Merge	Theta(n log(n))	Theta(n log(n))	Theta(n log(n))	Theta(n)	**Yes**	**No**

Three questions this section answers:

1. How can an elementary sort ever beat an advanced sort?
2. Can a comparison sort have a worst case better than Theta(n log(n))?
3. What does the STL use for `std::sort()`?

14.8.1 Performance of Elementary and Advanced Sorts

- Time complexity measures **how runtime scales**, not the actual runtime.
- An algorithm taking 2n steps is in the same class as one taking 200n steps, but the 2n one is much faster.
- Compare $2n^2$ against 200n log(n). The second has better complexity, but for **small n** the first is actually faster.
- This is exactly why **insertion sort beats Theta(n log(n)) sorts on small arrays**: it is simple and does little work per element. Doing more work per element only pays off once n is large.

14.8.2 Can Comparison Sorts Beat Theta(n log(n))?

No. If you can only use comparisons, you must make Omega(n log(n)) comparisons in the worst case.

- Suppose comparator `comp(x, y)` returns true if x <= y and false if x > y. Given three values a, b, c, the comparisons form a **decision tree**:

```
                  comp(a, b)
           T /                 \ F
        comp(b, c)             comp(a, c)
     T /      \ F           T /       \ F
  [a, b, c]   comp(a, c)  [b, a, c]   comp(b, c)
           T /     \ F              T /      \ F
        [a, c, b]  [c, a, b]     [b, c, a]  [c, b, a]
```

- With 3 values there are 3! = 6 possible permutations, shown at the leaves. You walk down the tree until one permutation remains.
- **Each comparison eliminates roughly half of the remaining permutations.** For example, `comp(a, b) == true` eliminates [b,a,c], [b,c,a], and [c,b,a].
- With n elements there are **n!** possible orderings. The number of comparisons x needed in the worst case satisfies:

```
n! / 2^x = 1     =>     x = log2(n!)
```

- From chapter 4, **log(n!) = Theta(n log(n))**.
- Therefore the worst-case time complexity of **any** comparison sort cannot be better than **Theta(n log(n))**.

14.8.3 Introsort and STL Sorting Implementations

- Many C++ standard library implementations use **introsort** (introspective sort) for `std::sort()`.
- How introsort works:
  - Starts with **quicksort**.
  - Switches to **heapsort** if the recursion depth gets too high, which prevents quicksort's Theta(n^2) worst case.
  - Switches to **insertion sort** when a partition is very small (typically fewer than 16 elements), since insertion sort is fastest there.
- Result: **Theta(n log(n)) average and worst case**.
- Introsort is the ideal example of an **adaptive** sort, because it uses many factors to decide which algorithm to use at any moment.
- Introsort relies on non-stable sorts, so it is **not stable**. Use `std::stable_sort()` if stability is needed. The STL typically implements `std::stable_sort()` with **mergesort**, since it is one of the few advanced comparison sorts that preserves relative order.

14.9 Counting Sort

- **Linear-time sorts** can beat Theta(n log(n)), but they require specific conditions on the input, so they are not always applicable.
- **Counting sort** works when the data values fall within a **limited set of keys**. It counts how many times each key appears, then uses arithmetic to compute where each key belongs in the output.

Steps

1. **First pass:** iterate over the data, counting occurrences of each key.
2. **Second pass:** convert the counts into **offsets** by taking a cumulative (prefix) sum, so each entry holds its own count plus all counts before it.
3. **Third pass:** iterate over the original data **in reverse** (this is what makes counting sort stable). For each value:
   - look up its offset k,
   - write the value to index k-1 of the output vector,
   - decrement k in the offset vector.

Example: grades, keys A B C D E F only. Data: `D B A C B F A B`
Counts: A=2 B=3 C=1 D=1 E=0 F=1
Offsets (prefix sums): A=2, B=2+3=5, C=6, D=7, E=7, F=8

Now traverse the data in reverse: B, A, F, B, C, A, B, D

```
B: offset 5 → write at index 4, B becomes 4  → [_,_,_,_,B,_,_,_]
A: offset 2 → write at index 1, A becomes 1  → [_,A,_,_,B,_,_,_]
F: offset 8 → write at index 7, F becomes 7  → [_,A,_,_,B,_,_,F]
B: offset 4 → write at index 3, B becomes 3  → [_,A,_,B,B,_,_,F]
C: offset 6 → write at index 5, C becomes 5  → [_,A,_,B,B,C,_,F]
A: offset 1 → write at index 0, A becomes 0  → [A,A,_,B,B,C,_,F]
B: offset 3 → write at index 2, B becomes 2  → [A,A,B,B,B,C,_,F]
D: offset 7 → write at index 6, D becomes 6  → [A,A,B,B,B,C,D,F]   sorted
```

```
void counting_sort(std::vector<char>& grades) {
  size_t num_keys = 6;                      // six possible grade values
  std::vector<char> output(grades.size());
  std::vector<size_t> offsets(num_keys);
  // first pass: count
  for (auto it = grades.begin(); it != grades.end(); ++it) {
    ++offsets[static_cast<size_t>(*it - 'A')];   // A = 0, B = 1, ...
  } // for it
  // second pass: cumulative sum for offsets
  for (size_t i = 1; i < num_keys; ++i) {
    offsets[i] += offsets[i - 1];
  } // for i
  // third pass (reverse): write to output
  for (auto it = grades.rbegin(); it != grades.rend(); ++it) {
    output[--offsets[static_cast<size_t>(*it - 'A')]] = *it;
  } // for it
  std::swap(grades, output);
} // counting_sort()
```

Complexity

- Let n = size of the data vector, **k = number of distinct keys** (size of the offset vector).
- Two traversals of the data and one of the offset vector: 2n + k = **Theta(n + k)** time in all cases.
- Allocates an output vector of size n and an offset vector of size k: **Theta(n + k)** auxiliary space.
- **Stable:** yes (thanks to the reverse third pass). **Adaptive:** no.

In-place variation

- Usable when the objects being counted **are** the objects being sorted. Count each key, then overwrite the original vector directly with the right number of each key.
- Uses **less auxiliary space**, but does not work in all situations depending on the object type, and it is **not stable**.
- Stability matters here: **radix sort depends on a stable counting sort**, so the in-place version cannot be used for it.

```
void in_place_counting_sort(std::vector<char>& grades) {
  size_t num_keys = 6;
  std::vector<size_t> counters(num_keys);
  // count the number of times each grade appears
  for (char grade : grades) {
    ++counters[static_cast<size_t>(grade - 'A')];
  } // for grade
  // write the correct number of each key back out
  size_t current = 0;
  for (size_t i = 0; i < num_keys; ++i) {
    for (size_t j = 0; j < counters[i]; ++j) {
      grades[current++] = static_cast<char>('A' + i);
    } // for j
  } // for i
} // in_place_counting_sort()
```

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(n + k)	Theta(n + k)	Theta(n + k)	Theta(n + k)	**Yes**	**No**

14.10 Radix Sort BONUS, NOT TESTED

Per course directions: section 14.10 is bonus material after the 2022 edit. Read it once for awareness, do not stress over it.

- **Radix sort** is a linear-time sort for **numeric values or fixed-size strings**.
- Digits or characters are grouped by their **significance position**. Then **counting sort** is applied repeatedly, from the **least significant** position to the **most significant**.
- Counting sort must be **stable**, so that the relative order established by earlier (less significant) passes is preserved.

Example on [659, 424, 953, 139, 380, 811, 187, 354, 769, 415, 286, 331]

```
start   : 659 424 953 139 380 811 187 354 769 415 286 331
by ones : 380 811 331 953 424 354 415 286 187 659 139 769
by tens : 811 415 424 331 139 953 354 659 769 380 286 187
by 100s : 139 187 286 331 354 380 415 424 659 769 811 953   sorted
```

Note 659 comes before 139 in the input, and stability keeps 659 before 139 after the ones-digit pass (both have 9 in the ones place).

- **Complexity Theta(d(n + k))** where d = number of digits in the largest value, n = input size, k = number of distinct keys. For integers, k is the **base** the input is expressed in (base 10 above).
- Each counting sort pass is Theta(n + k), and d passes are needed.
- **Auxiliary space Theta(n + k)**, same as counting sort.

Footnote: starting at the least significant digit is called **LSD radix sort**. You can also start at the most significant digit, which is called **MSD radix sort**.

Best Time	Average Time	Worst Time	Auxiliary Space	Stable?	Adaptive?
Theta(d(n+k))	Theta(d(n+k))	Theta(d(n+k))	Theta(n + k)	**Yes**	**No**

14.11 Index Sorting

Index sorting: accessing the elements of a container in sorted order **without actually sorting the container's data**. Instead you sort a separate container of **indices** that acts as a proxy for the real data.

When it is useful

- You want the sorted order but must **preserve the original ordering** of the data.
- The container holds **large objects** that are too expensive to move around.

How the index vector is ordered

- The indices are **not** sorted ascending by their own values. Instead, index `i` comes before index `j` if `vec[i] < vec[j]` in the original data.

Example

```
vec  :  7.5  4.4  6.7  3.2  5.5  1.3  9.6  8.3
index:   0    1    2    3    4    5    6    7

Start with idx = [0, 1, 2, 3, 4, 5, 6, 7].
Smallest value is 1.3 at index 5, so 5 goes first in idx.
Next smallest is 3.2 at index 3, so 3 goes second.
Continuing: idx = [5, 3, 1, 4, 2, 0, 7, 6]
```

After the index sort, the smallest element of vec is `vec[idx[0]]`, the second smallest is `vec[idx[1]]`, and so on. `vec` itself is untouched.

```
// print all elements of vec in sorted order
for (size_t i = 0; i < idx.size(); ++i) {
  std::cout << vec[idx[i]] << '\n';
} // for i
```

Implementing it with a custom comparator

- `std::sort()` accepts an optional comparator:
  `void std::sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);`
- The comparator must know the **original data** to decide how to order the indices. Store that data in the comparator's **internal state**.
- Store it as a **reference**, not a copy, so you do not duplicate a potentially huge container every time a comparator is constructed.
- References must be initialized when created, so use a constructor with an **initializer list**.

```
template <typename T>
class IndexSortComparator {
  const std::vector<T>& data;             // reference to the original data
public:
  IndexSortComparator(const std::vector<T>& vec)
    : data{vec} {}
  ...
};
```

- A normal ascending comparator on integers would be:

```
bool operator() (size_t i, size_t j) const {
  return i < j;
} // operator()
```

- But for an index sort we do **not** compare i and j themselves. We compare the elements **at** positions i and j in the underlying data:

```
bool operator() (size_t i, size_t j) const {
  return data[i] < data[j];
} // operator()
```

Full comparator

```
template <typename T>
class IndexSortComparator {
  const std::vector<T>& data;
public:
  IndexSortComparator(const std::vector<T>& vec)
    : data{vec} {}
  bool operator() (size_t i, size_t j) const {
    return data[i] < data[j];
  } // operator()
};
```

This requires that type T supports `operator<`.

Putting it together

```
std::vector<double> vec = {7.5, 4.4, 6.7, 3.2, 5.5, 1.3, 9.6, 8.3};
// initialize vector of indices
std::vector<size_t> idx(vec.size());
// fill idx with 0, 1, 2, ...
std::iota(idx.begin(), idx.end(), 0);
// construct comparator with a reference to vec
IndexSortComparator<double> idx_comp{vec};
// index sort
std::sort(idx.begin(), idx.end(), idx_comp);
// idx is now {5, 3, 1, 4, 2, 0, 7, 6}
```